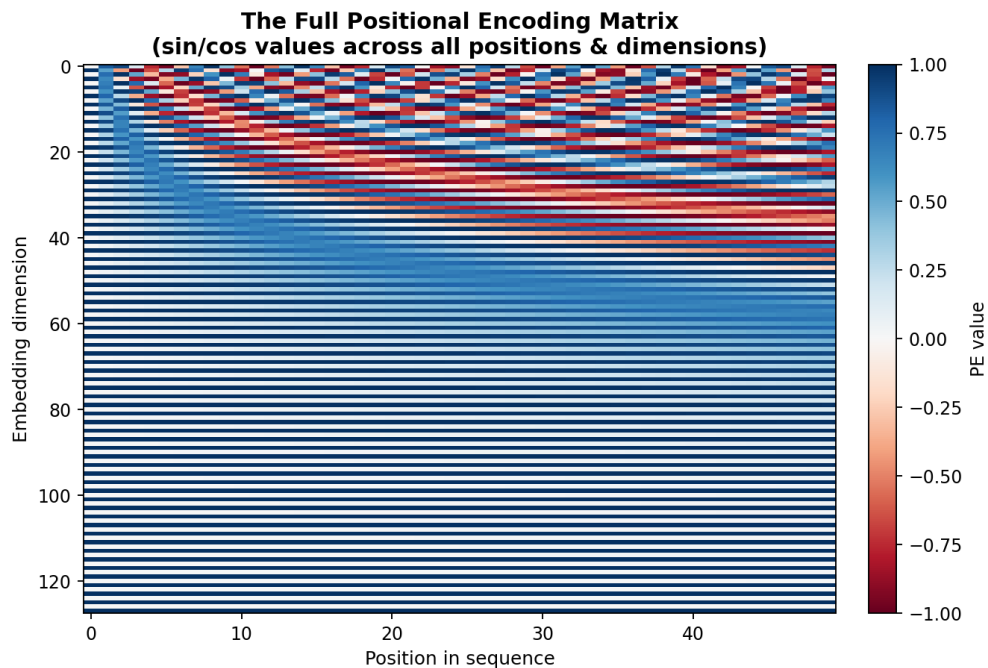


Positional Encoding in Transformers

Why It's Required, the Simple Solutions, and the Mathematical Intuition Behind Sine/Cosine



Covers: Why positional encoding is required • Proposing simple solutions • The sine function as the solution • Mathematical intuition

1. Why Is Positional Encoding Required?

Unlike RNNs, which process words one at a time in order (naturally encoding position through the sequence of computation), the Transformer processes ALL words simultaneously via self-attention. Self-attention computes a weighted sum over Value vectors based purely on content similarity (Query · Key) - it has **no built-in sense of order** at all.

Self-Attention Treats Input as a SET, Not a Sequence



Figure 1: Self-attention operates on the input as an unordered SET - shuffle the words, and without position info, the output stays the same.

This creates a real problem, since word order carries meaning:

Without Positional Info, Self-Attention Can't Tell These Apart

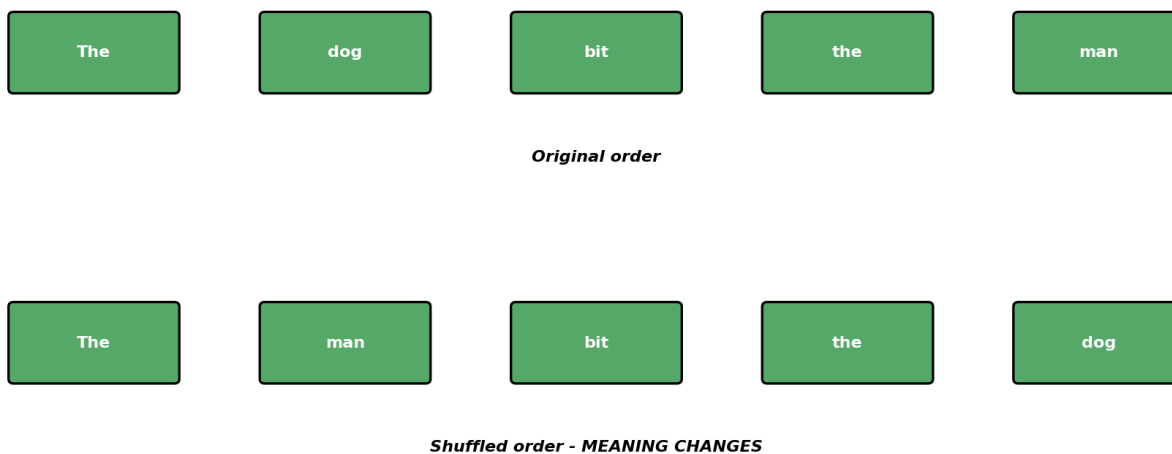


Figure 2: "The dog bit the man" and "The man bit the dog" contain the exact same words - only their order differs, and yet the meaning is completely different.

Without some way of injecting order information, a Transformer would treat these two sentences as identical. **Positional encoding** is the mechanism that fixes this - it gives the model a way to know not just WHAT each word is, but WHERE it sits in the sequence.

2. Proposing a Simple Solution

Before landing on sine/cosine, it's worth walking through the more obvious, naive ideas first - and seeing exactly why each one falls short.

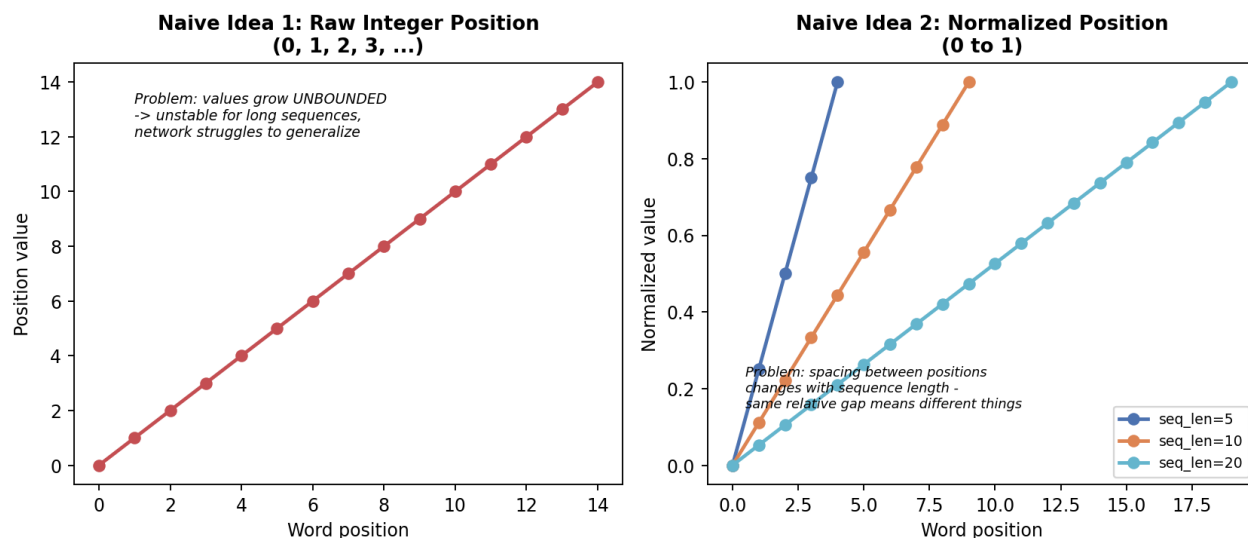


Figure 3: Two naive attempts at encoding position, and the problems each one runs into.

Naive Idea 1: Just Use the Raw Position Number

Simplest possible idea: assign position 0 to the first word, 1 to the second, 2 to the third, and so on, then add this number directly to the embedding.

- **Problem:** for long sequences, these values grow unbounded (position 5000 vs position 3) - very large numbers destabilize training and dominate the embedding values they're added to.
- **Problem:** the model never saw such large position values during training on shorter sequences, so it generalizes poorly to new lengths.

Naive Idea 2: Normalize Position Between 0 and 1

A fix for unbounded growth: divide each position by the sequence length, so values always fall between 0 and 1.

- **Problem:** the actual gap between positions now depends on sequence length. Position 0.5 in a 10-word sentence is a completely different absolute position than 0.5 in a 100-word sentence - the encoding is inconsistent across different sequence lengths.

What we actually need is an encoding that is **bounded** (doesn't blow up), **consistent** regardless of sequence length, **unique** for every position, and ideally lets the model easily reason about relative distances between positions. This is exactly what the sine/cosine solution achieves.

3. The Sine Function as a Simple Solution

The Transformer paper's solution: use sine and cosine functions of different frequencies, one pair per pair of embedding dimensions. Each dimension of the positional encoding vector oscillates at a different frequency - some change quickly from word to word, others change very slowly across long spans of the sequence.

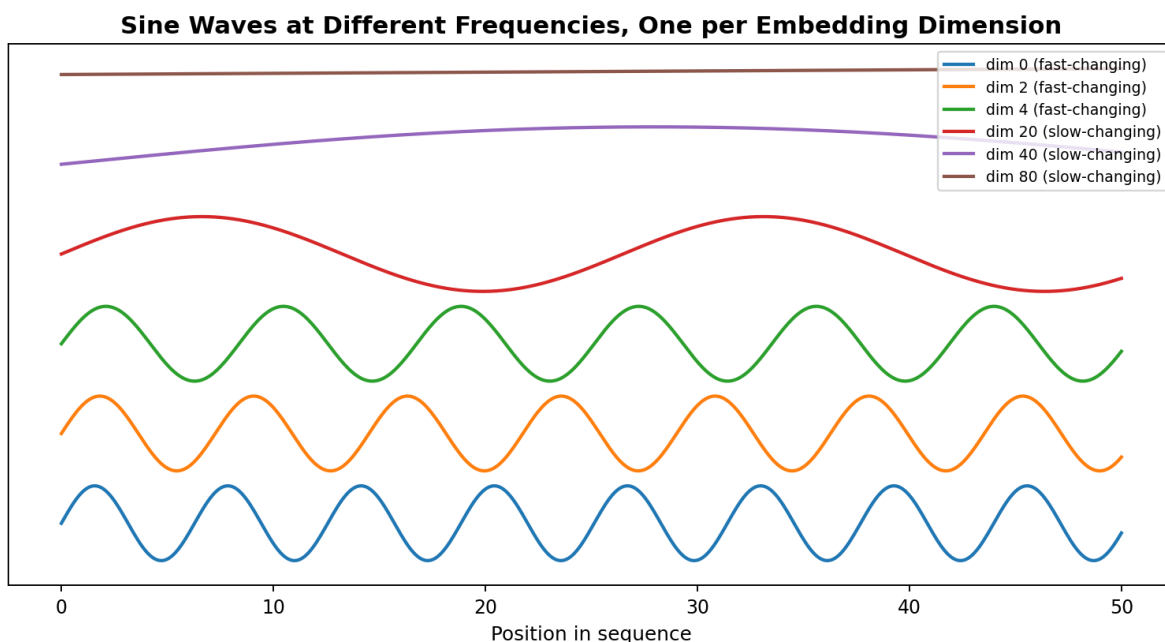


Figure 4: Different embedding dimensions use different wave frequencies - like a set of clocks ticking at different speeds.

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d_model)})$$

$$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d_model)})$$

Here, **pos** is the word's position in the sequence, and **i** indexes the embedding dimension pair. Even dimensions use sine, odd dimensions use cosine, and the frequency decreases geometrically as **i** increases (from very fast-changing early dimensions to very slow-changing later dimensions).

Putting every position and every dimension together produces a full matrix of values - this is the actual positional encoding added to every word embedding:

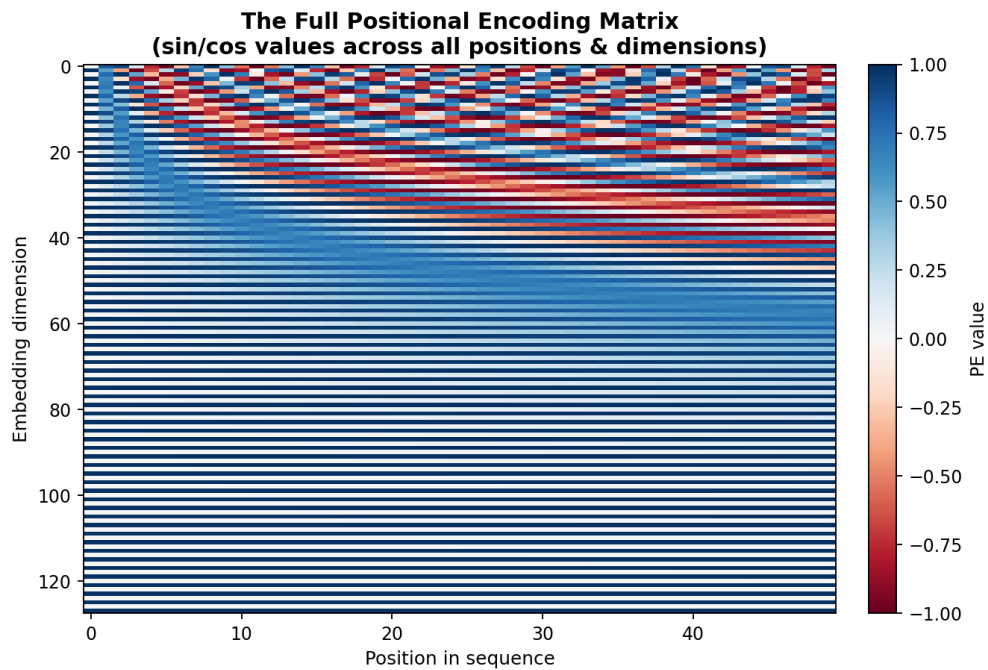


Figure 5: The complete positional encoding matrix. Each column is one position's unique 'fingerprint' across all embedding dimensions.

This matrix is computed once, has exactly the same shape as the word embedding matrix, and is simply **added** element-wise to the word embeddings before the first Transformer layer:

Positional Encoding Is ADDED to Word Embeddings

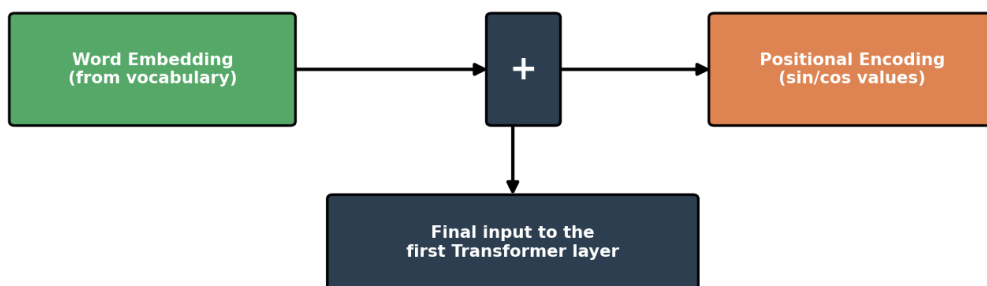


Figure 6: Positional encoding is added directly to the word embedding - not concatenated, not learned as a separate lookup, just summed in.

4. Mathematical Intuition - Why Sine/Cosine, Specifically?

It's worth digging a little deeper into why THIS particular function was chosen, rather than any other bounded, periodic function. There are a few properties that make sine/cosine especially well-suited to this job.

Why Sine/Cosine Works So Well

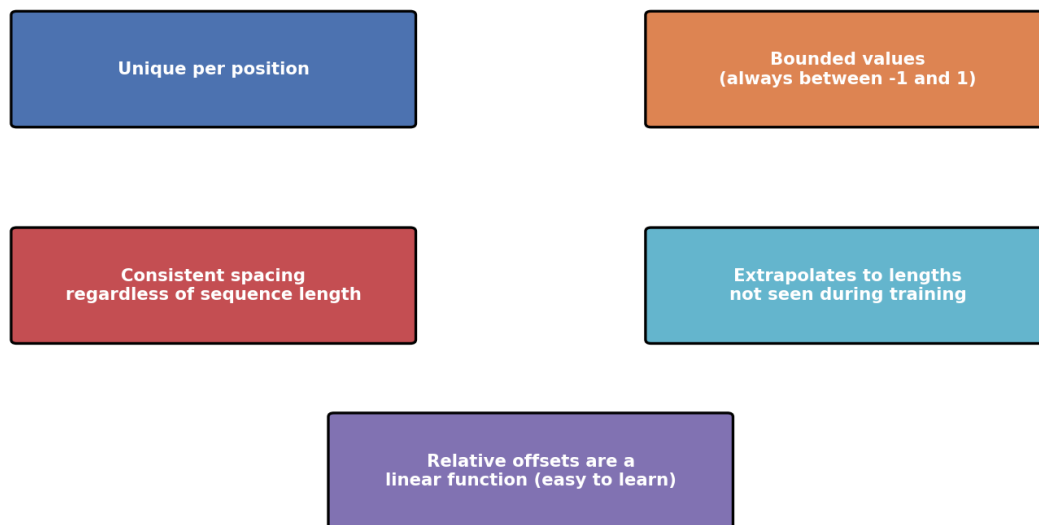


Figure 7: The five key properties that make sinusoidal positional encoding effective.

Property 1 & 2: Unique and Bounded

Because each dimension oscillates at a different frequency, the combination of values across all dimensions gives every position a distinct "fingerprint" - much like how a combination of different clock hands (seconds, minutes, hours) uniquely identifies a moment in time. And because sine and cosine are always between -1 and 1, values never blow up, no matter how long the sequence gets.

Property 3: Consistent Spacing

Unlike normalized position (0 to 1), sine/cosine values for a given position don't depend on the total sequence length at all - position 5 always gets the exact same encoding, whether the sentence has 10 words or 10,000.

Property 4: Extrapolation to Unseen Lengths

Because the encoding is a mathematical formula (not a learned lookup table), it can be computed for ANY position, even ones longer than any sequence seen during training - giving the model at least some ability to handle longer inputs than it was trained on.

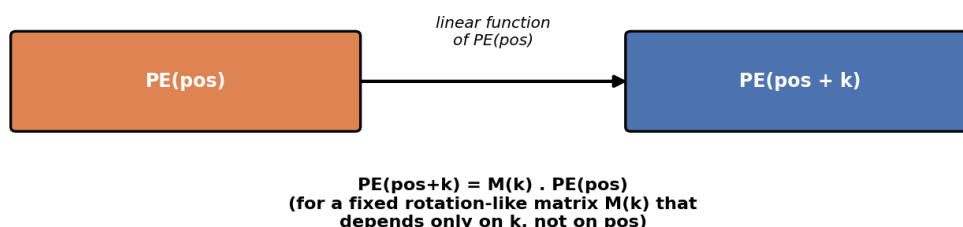
Property 5: Relative Positions Are a Linear Function (The Key Insight)

This is the most elegant property, and the real reason sine/cosine PAIRS were chosen (rather than sine alone). Using the trigonometric angle-addition identities:

$$\sin(a + b) = \sin(a)\cos(b) + \cos(a)\sin(b) \quad \cos(a + b) = \cos(a)\cos(b) - \sin(a)\sin(b)$$

it turns out that $PE(pos + k)$ can be written as a **fixed linear transformation** of $PE(pos)$, for any offset k - and crucially, that linear transformation depends only on k , NOT on pos itself.

Mathematical Intuition: Relative Positions Are Linearly Related



This is exactly why sin/cos was chosen: trig identities ($\sin(a+b)$, $\cos(a+b)$) make relative offsets easy for the model to learn

Figure 8: For any fixed offset k , there's a consistent linear relationship between $PE(pos)$ and $PE(pos+k)$ - regardless of where 'pos' actually is.

In practice, this means a linear layer inside the Transformer (like the ones used in attention) can learn to detect relative positions - "this word is 3 positions after that one" - using simple linear operations, rather than needing to learn some complicated, position-specific rule from scratch. This is a big part of why sinusoidal encoding works so well as a simple, non-learned solution.

Summary

Concept	Key Idea
Why needed	Self-attention has no built-in sense of word order
Naive idea: raw position	Unbounded values, unstable for long sequences
Naive idea: normalized position	Spacing changes with sequence length - inconsistent
The sine/cosine solution	sin/cos at different frequencies per embedding dimension
How it's applied	Added element-wise to word embeddings, once, before layer 1

Key properties	Unique, bounded, length-consistent, extrapolates, relative offsets are linear
----------------	---

Positional encoding is a small addition, computationally speaking, but it's essential: without it, the Transformer's biggest strength - fully parallel, order-agnostic self-attention - would also be its biggest weakness. Sine and cosine functions give the model a cheap, elegant, non-learned way to recover exactly the ordering information it would otherwise lose.